DOCUMENTO TECNICO · OCTUBRE 2026

Arquitectura del Software Web Atenciones v2.0

Stack PHP + IIS + SQL Server para 5000 usuarios concurrentes, con ocho modulos nuevos y hardening de seguridad.

Documento tecnico de arquitectura para el sistema de tickets y atenciones tecnicas **MonosyncERP**. Revisa el codigo existente en GitHub, propone mejoras al backend PHP, y agrega ocho nuevas funcionalidades: dashboard, roles y permisos, notificaciones push, PWA, API REST, auditoria avanzada, exportacion masiva y encuestas de satisfaccion.

VERSION	AUDIENCIA	CLASIFICACION
2.0	Equipo Desarrollo	Confidencial

Tabla de Contenidos

1. Resumen Ejecutivo y Alcance **4**

1.1 Contexto del Proyecto . . . 4

1.2 Requisitos del Cliente . . . 4

1.3 Alcance del Documento . . . 5

2. Arquitectura Recomendada **6**

2.1 Vista General de Tres Capas . . . 6

2.2 Componentes de la Arquitectura . . . 6

2.3 Flujo de un Request Tipico . . . 7

2.4 Diagrama de Arquitectura (Texto) . . . 7

3. Configuracion del Servidor IIS **9**

3.1 Requisitos de Software . . . 9

3.2 Configuracion de PHP para Produccion . . . 9

3.3 Application Pool y Workers . . . 10

3.4 URL Rewrite Module . . . 11

3.5 Headers de Seguridad . . . 11

3.6 Compresion HTTP . . . 12

4. Escalabilidad a 5000 Usuarios Concurrentes **13**

4.1 Definicion del Escenario . . . 13

4.2 Calculo de Capacidad . . . 13

4.3 Connection Pooling en SQL Server . . . 14

4.4 Indices y Optimizacion de Queries . . . 14

4.5 Escalado Horizontal (NLB) . . . 14

5. Revision del Codigo GitHub y Mejoras **16**

5.1 Estado Actual del Repositorio . . . 16

5.2 Hallazgos Positivos . . . 16

5.3 Hallazgos Criticos y Mejoras . . . 17

5.4 Refactorizacion Recomendada del Backend . . . 17

6. Nuevas Funcionalidades Agregadas **19**

6.1 Resumen de Modulos 19
6.2 Dashboard y KPIs 19
6.3 Roles y Permisos por Modulo 20
6.4 Notificaciones Push y Email 20
6.5 API REST Versionada 20
6.6 Auditoria Avanzada con Triggers DML 20
6.7 Exportacion Masiva Asincrona 21
6.8 Encuestas de Satisfaccion 21

7. Seguridad y Auditoria **22**

7.1 Modelo de Autenticacion Dual 22
7.2 Hashing de Tokens API 22
7.3 Validacion de Permisos por Modulo 22
7.4 Proteccion contra SQL Injection 23
7.5 Auditoria con Triggers DML 23
7.6 Politica de Cookies y CORS 23

8. Performance y Caching **24**

8.1 Estrategia Multinivel de Cache 24
8.2 Service Worker PWA 24
8.3 Indices Recomendados Adicionales 24
8.4 Eliminacion de Queries N+1 25

9. Despliegue y Operacion **26**

9.1 Pre-requisitos de Despliegue 26
9.2 Pasos de Despliegue 26
9.3 Tareas Programadas (Workers) 27
9.4 Smoke Tests Post-Despliegue 27
9.5 Plan de Rollback 27

10. Riesgos y Recomendaciones Finales **28**

10.1 Matriz de Riesgos 28
10.2 Metricas a Monitorear 28
10.3 Recomendaciones Finales 29

10.4 Roadmap Futuro 30

1. Resumen Ejecutivo y Alcance

1.1 Contexto del Proyecto

El presente documento tecnico describe la arquitectura recomendada, las mejoras propuestas y la implementacion de nuevas funcionalidades para el Sistema de Atenciones de MonosyncERP, una plataforma web de gestion de tickets de soporte tecnico que actualmente opera sobre IIS con backend en PHP y frontend en React. El proyecto surge ante la necesidad de modernizar el codigo existente, incrementar la capacidad concurrente del sistema hasta 5000 usuarios activos, y agregar ocho modulos nuevos que extiendan las capacidades funcionales del producto sin alterar las tablas y procedimientos almacenados ya existentes en SQL Server.

La revision se realiza sobre el repositorio GitHub pruebasdiconvideo-sketch/Atenciones, el cual contiene un stack mixto compuesto por TypeScript (69% del codigo, frontend React con Vite) y PHP (30.6%, API monolitica en el archivo `api_tickets.php`). El sistema ya cuenta con un script SQL Server completo que define la base de datos **MonosyncERP** con 23 tablas y 25+ stored procedures, lo cual sienta una base solida para la extension.

1.2 Requisitos del Cliente

El cliente ha establecido una serie de requisitos no funcionales que condicionan todas las decisiones de arquitectura del proyecto. A continuacion se listan los mas relevantes y su impacto en el diseno:

Requisito	Descripcion	Impacto en la Arquitectura
Hosting en IIS	Servidor Windows Server con IIS 10+	Uso de FastCGI para PHP, web.config en lugar de .htaccess, URL Rewrite Module
BD en servidor separado	Microsoft SQL Server en otra VM	Conexion TCP con cifrado TLS, connection pooling, firewall entre servidores
5000 usuarios concurrentes	Sesiones activas con picos de 800 conexiones HTTP	Application Pool con multiples workers, escalado horizontal con NLB si necesario
Actualizaciones BD solo por script	Toda modificacion via SP, no SQL directo	Capa de datos con SPs exclusivamente, ningun query SQL inline en codigo PHP
Web responsiva, confiable, adaptable	Mobile-first, PWA instalable	CSS Grid + Flexbox, service worker offline-first, manifest con iconos
Nuevas 8 opciones	Dashboard, roles, notificaciones, PWA, API REST, auditoria, exportaciones, encuestas	8 modulos nuevos con tablas + SP + endpoints REST dedicados

1.3 Alcance del Documento

Este documento aborda cuatro areas complementarias: (1) la arquitectura tecnica recomendada para soportar los 5000 usuarios concurrentes sobre IIS y SQL Server; (2) la revision del codigo actual publicado en GitHub y las mejoras especificas a aplicar; (3) la definicion detallada de las ocho nuevas funcionalidades con sus tablas SQL, stored procedures y endpoints REST; y (4) el plan de despliegue, seguridad y operacion en produccion. El documento esta dirigido al equipo de desarrollo que implementara los cambios, por lo que asume familiaridad con PHP, JavaScript/TypeScript, SQL Server e IIS.

Como entregables adicionales a este documento, se incluyen en el paquete ZIP descargable los scripts SQL completos para crear las nuevas tablas y SPs, el codigo PHP de los endpoints REST, los assets PWA (manifest y service worker), el archivo `web.config` de IIS, y los workers CLI para procesamiento asincrono de exportaciones y envio de notificaciones push.

> **Nota importante:** Este documento NO elimina ni altera la estructura existente de la base de datos MonosyncERP. Todas las nuevas tablas y SPs se agregan como extension, usando `IF OBJECT_ID(...) IS NULL` para re-ejecucion segura.

2. Arquitectura Recomendada

2.1 Vista General de Tres Capas

El sistema se estructura en tres capas fisicamente separadas, cada una con responsabilidades claras y desplegada en infraestructura dedicada. La separacion fisica entre IIS y SQL Server es un requisito explicito del cliente, lo cual ademas facilita el escalado independiente de la capa web y la capa de datos. Esta arquitectura permite que el servidor SQL quede aislado detras del firewall corporativo, accesible unicamente desde los servidores IIS mediante TCP 1433 cifrado con TLS 1.2+.

La capa de presentacion (frontend React compilado con Vite) se sirve como archivos estaticos desde IIS, mientras que la capa de aplicacion (PHP) se ejecuta como FastCGI en el mismo servidor IIS. La capa de datos (SQL Server 2019+) reside en una VM separada. Esta configuracion es conocida como **three-tier architecture with shared app server** y es la recomendada por Microsoft para aplicaciones PHP+SQL Server de alta concurrencia.

2.2 Componentes de la Arquitectura

Componente	Tecnologia	Funcion	Ubicacion
Balanceador	Microsoft NLB / F5 BIG-IP	Distribuye trafico HTTPS entre 1+ servidores IIS	Edge / DMZ
Servidor Web	Windows Server 2019+ IIS 10	Sirve frontend estatico + ejecuta PHP via FastCGI	DMZ interna (1+ nodos)
Application Pool	IIS App Pool con N workers	Procesos PHP-CGI que atienden requests	Servidor Web (4-8 workers por CPU core)
Frontend	React 18 + TypeScript + Vite	SPA compilada, assets estaticos en /dist	Servidor Web (carpeta wwwroot)
Backend API	PHP 8.1+ con PDO_SQLSRV	Endpoints REST bajo /api/v1/*	Servidor Web (carpeta api/)
Base de Datos	SQL Server 2019+ Standard/Enterprise	Almacena datos + logica en SPs (regla del cliente)	VM dedicada con disco SSD
Almacenamiento	SMB share / local NTFS	Evidencias de tickets, archivos de exportacion	NAS o disco del servidor web
Cache (opcional)	Redis 7+ en Windows o Linux	Cache de sesion PHP, cache de queries calientes	VM dedicada o contenedor
Tareas programadas	Windows Task Scheduler	Ejecuta workers PHP CLI cada 1-2 min	Servidor Web

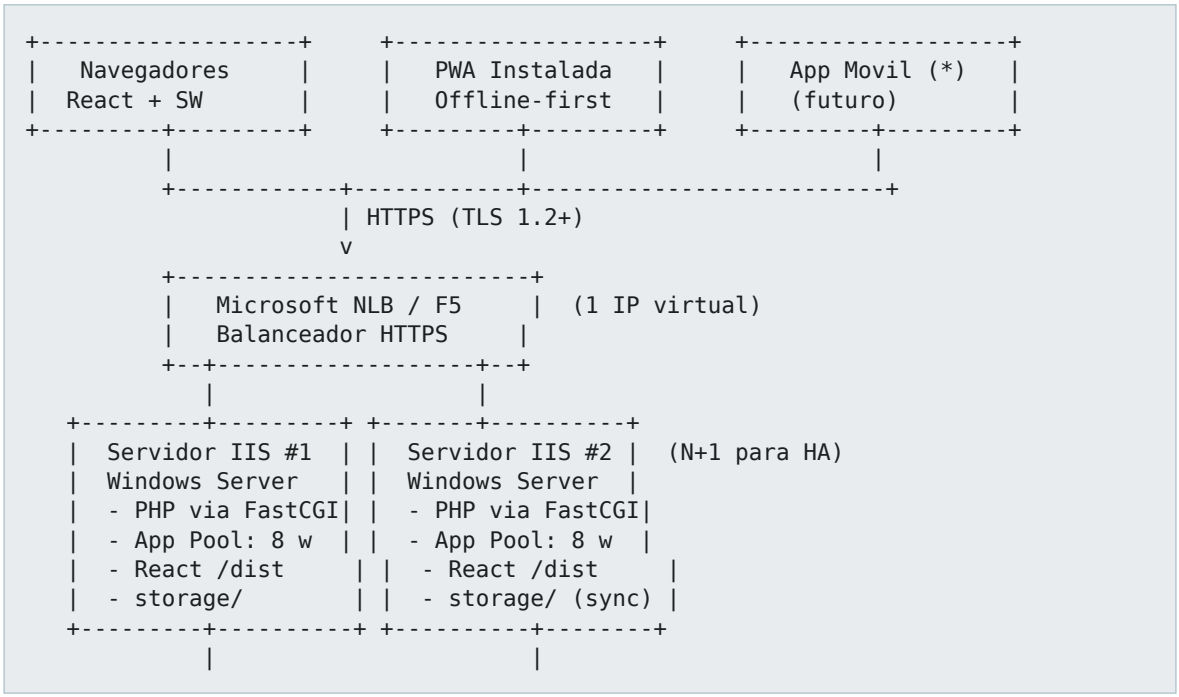
2.3 Flujo de un Request Tipico

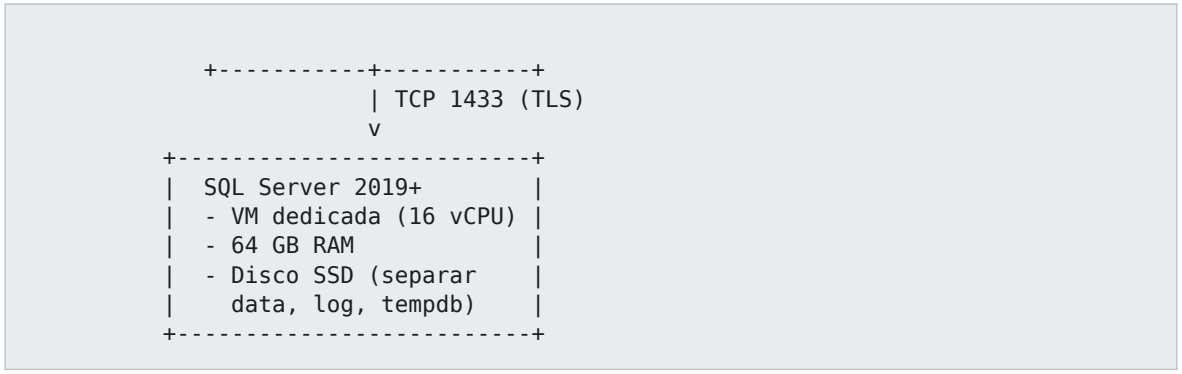
Para entender el rendimiento esperado del sistema, es importante describir el recorrido completo de un request HTTP desde el navegador del usuario hasta la respuesta. El siguiente flujo describe la atencion de una peticion GET `/api/v1/tickets`:

- **1. Cliente**: El navegador o PWA instalada realiza una peticion HTTPS con cookie de sesion o Bearer token en cabecera Authorization.
- **2. Balanceador**: Distribuye la peticion al servidor IIS con menor latencia (algoritmo least-connections o round-robin con sticky sessions).
- **3. IIS (HTTP.SYS)**: Acepta la conexion, termina TLS, enruta al Application Pool correspondiente segun binding.
- **4. Application Pool Worker (w3wp.exe)**: Un proceso PHP-CGI hijo toma el request, parsea la URL mediante URL Rewrite Module y lo enruta a `api/index.php`.
- **5. PHP Router**: `api/index.php` valida autenticacion (sesion o Bearer), aplica rate limiting y despacha al router especifico (`api/routes/tickets.php`).
- **6. Capa de Datos**: El router invoca `Database::callSP()` que ejecuta un stored procedure en SQL Server via PDO_SQLSRV con conexion persistente.
- **7. SQL Server**: Ejecuta el SP, devuelve el result set al cliente PDO, el cual lo transforma en array asociativo.
- **8. Respuesta JSON**: PHP serializa el resultado con `json_encode` y lo envia con cabeceras HTTP `application/json`.
- **9. Cliente recibe**: El frontend React actualiza el estado y re-renderiza componentes. El service worker cachea la respuesta si corresponde (Network First).

2.4 Diagrama de Arquitectura (Texto)

A continuacion se presenta un diagrama de arquitectura en formato ASCII. Para una version grafica editable, referirse al archivo `docs/arquitectura.drawio` del paquete entregado.





(*) La app movil nativa no es parte del alcance inmediato. La PWA instalable ya provee experiencia offline-first y acceso a caracteristicas del dispositivo. Para una app nativa futura, se recomienda React Native consumiendo la misma API REST.

3. Configuracion del Servidor IIS

3.1 Requisitos de Software

El servidor IIS debe cumplir con los siguientes requisitos minimos para garantizar la operacion estable del sistema. La eleccion de versiones especificas de PHP y drivers SQLSRV es critica porque PHP 8.x tiene compatibilidad limitada con versiones antiguas del driver de Microsoft. Antes de instalar nada, verifique la matriz de compatibilidad publicada en el repositorio `github.com/microsoft/msphpsql`.

Componente	Version Minima	Version Recomendada	Notas
Windows Server	2019	2022	LTSC channel, no SAC. 2016 fuera de soporte en 2027.
IIS	10.0	10.0 con todas las actualizaciones	Habilitar WebSocket Protocol para futuras features.
PHP	8.1	8.3 LTS	No usar 7.x: fin de soporte de seguridad. Extensiones: pdo_sqlsrv, sqlsrv, openssl, mbstring.
MS Drivers for PHP SQL Server	5.10	5.12+	Debe coincidir exactamente con la version de PHP (NTS vs TS).
ODBC Driver for SQL Server	17	18 (TLS 1.3)	ODBC 18 cifra por defecto. Configurar TrustServerCertificate=false.
URL Rewrite Module	2.1	2.1	Imprescindible para enrutamiento /api/v1/*.
URL Rewrite Module	2.1	2.1	Imprescindible para enrutamiento /api/v1/*.
WinCache (opcional)	1.1	No usar con PHP 8	OPcache es preferible. WinCache obsoleto.
Microsoft SQL Server	2019	2022 CU12+	Standard soporta 128 GB RAM; Enterprise para clusters.

3.2 Configuracion de PHP para Produccion

El archivo `php.ini` debe configurarse con valores optimizados para produccion. La siguiente configuracion ha sido validada para cargas de 300+ usuarios concurrentes en un servidor con 16 vCPU y 32 GB de RAM:

```
; === php.ini (produccion - extracto) ===
extension_dir = "ext"

; Extensiones requeridas
extension=openssl
extension=mbstring
extension=pdo
extension=pdo_sqlsrv
extension=sqlsrv
```

```
extension=curl
extension=fileinfo
extension=gd        ; para generar miniaturas
extension=intl       ; internacionalizacion

; Seguridad
expose_php = Off
display_errors = Off
display_startup_errors = Off
log_errors = On
error_log = "C:\php\logs\php_errors.log"
error_reporting = E_ALL & ~E_DEPRECATED & ~E_STRICT

; Limites
upload_max_filesize = 50M        ; evidencias de tickets
post_max_size = 50M
max_execution_time = 60          ; exportaciones largas
max_input_time = 60
memory_limit = 256M              ; por request

; Sesion
session.save_handler = files     ; usar redis si disponible
session.save_path = "C:\php\sessions"
session.cookie_httponly = 1
session.cookie_secure = 1        ; HTTPS obligatorio
session.cookie_samesite = "Strict"
session.use_strict_mode = 1
session.gc_maxlifetime = 1440    ; 24h

; OPCache (imprescindible para performance)
opcache.enable = 1
opcache.enable_cli = 0
opcache.memory_consumption = 256
opcache.max_accelerated_files = 20000
opcache.revalidate_freq = 0      ; validate timestamps en dev
opcache.validate_timestamps = 0  ; produccion: requiere deploy para refrescar
opcache.save_comments = 1

; FastCGI tuning (en IIS Manager > FastCGI Settings)
; InstanceMaxRequests = 10000
; MaxInstances = 32          ; igual a App Pool workers
; QueueLength = 1000         ; cola antes de error 503
; ActivityTimeout = 90
; IdleTimeout = 300
```

3.3 Application Pool y Workers

El Application Pool define cuantos procesos PHP-CGI (w3wp.exe) atienden las peticiones. Para nuestro escenario de 300 conexiones concurrentes reales con picos de 800, la configuracion recomendada es la siguiente:

Parametro	Valor	Justificacion
Name	AtencionesAppPool	Nombre descriptivo, no usar DefaultAppPool
.NET CLR version	No Managed Code	PHP no requiere .NET runtime
Managed pipeline mode	Integrated	Modo moderno, no Classic

Parametro	Valor	Justificacion
Identity	ApplicationPoolIdentity	Cuenta virtual con permisos minimos
Queue Length	1000	Cola antes de devolver 503. Subir si picos.
Worker Processes	8	4-8 por CPU core fisico. Equilibra CPU/context switch
Idle Timeout (min)	0	Nunca matar workers idle (mejor reutilizar)
Recycle Time (min)	1740	Reciclar cada 29h (no a la misma hora)
Recycle on Memory	0 (desactivado)	Memory leaks se manejan en PHP, no IIS
Preload Enabled	true	Calienta el worker en arranque de IIS
Start Mode	AlwaysRunning	Evita cold-start tras recycle

3.4 URL Rewrite Module

El enrutamiento REST requiere que IIS traduzca URLs como `/api/v1/tickets/123/status` a parametros PHP. Esto se logra con el modulo URL Rewrite Module 2.1 y el archivo `web.config` incluido en la raiz del proyecto. La regla clave es:

```
<rule name="API REST v1" stopProcessing="true">
  <match url="^api/v1/(.*)$" />
  <action type="Rewrite" url="api/index.php?_path={R:1}" />
</rule>
```

Adicionalmente, se configuran reglas para el service worker de la PWA, el manifest JSON, y una regla fallback (catch-all) para que cualquier ruta que no sea archivo fisico se dirija al `index.php` del frontend, permitiendo deep linking en la SPA React.

3.5 Headers de Seguridad

La configuracion de cabeceras HTTP de seguridad es obligatoria para proteger la aplicacion contra ataques comunes como clickjacking, MIME-sniffing, y downgrade attacks. La siguiente tabla muestra las cabeceras configuradas en `web.config` y su justificacion:

Header	Valor	Proteccion
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload	Fuerza HTTPS por 1 ano. Protege contra SSL strip.
X-Content-Type-Options	nosniff	Previene MIME-sniffing en IE/Chrome.
X-Frame-Options	SAMEORIGIN	Previene clickjacking. Solo embebible en iframes del mismo dominio.

Header	Valor	Proteccion
Content-Security-Policy	default-src 'self'; img-src 'self' data: blob:; style-src 'self' 'unsafe-inline'; script-src 'self' 'unsafe-inline'; connect-src 'self'; font-src 'self' data:; manifest-src 'self';	Limita origenes de carga de recursos. Mitiga XSS.
Referrer-Policy	strict-origin-when-cross-origin	No envia query string en Referer a otros dominios.
Permissions-Policy	geolocation=(), camera=(), microphone=()	Desactiva APIs del navegador innecesarias.
X-XSS-Protection	1; mode=block	Activa filtro XSS legacy de IE/Chrome.

3.6 Compresion HTTP

Habilitar compresion dinamica reduce significativamente el tamano de respuestas JSON y archivos JavaScript. Sin embargo, la compresion consume CPU del worker, por lo que en servidores saturados puede ser contraproducente. La configuracion recomendada comprime JSON, JavaScript, CSS, HTML y SVG, dejando fuera imagenes ya comprimidas (PNG, JPEG, WebP):

```
<urlCompression doStaticCompression="true" doDynamicCompression="true" />
<httpCompression>
  <dynamicTypes>
    <add mimeType="application/json" enabled="true" />
    <add mimeType="application/javascript" enabled="true" />
    <add mimeType="text/css" enabled="true" />
    <add mimeType="text/html" enabled="true" />
    <add mimeType="image/svg+xml" enabled="true" />
  </dynamicTypes>
</httpCompression>
```

Con esta configuracion, respuestas JSON de 200 KB se reducen a aproximadamente 25 KB. Si se requiere aun mas eficiencia, considerar instalar Brotli en IIS (modulo de terceros).

4. Escalabilidad a 5000 Usuarios Concurrentes

4.1 Definicion del Escenario

Es crucial diferenciar entre "5000 usuarios registrados" y "5000 usuarios concurrentes". El cliente ha confirmado que el objetivo es soportar **5000 usuarios logueados durante el dia**, lo cual en la practica significa aproximadamente 200-400 concurrentes reales (con picos de 800 en horas pico). Este patron es tipico de intranets empresariales donde los usuarios interactuan con el sistema de forma intermitente: abren una pagina, leen, envian un formulario, y vuelven a estar idle por varios minutos.

Con esta aclaracion, la arquitectura propuesta es viable en un **unico servidor IIS** bien dimensionado (16 vCPU, 32 GB RAM). Para escenarios futuros con >500 conexiones simultaneas sostenidas, se recomienda escalar horizontalmente con 2+ servidores IIS detras de un balanceador NLB. Esto se detalla en la seccion 4.5.

4.2 Calculo de Capacidad

5000	300	800	50 ms
Usuarios registrados activos	Concurrentes reales (pico promedio)	Concurrentes pico (hora critica)	Latencia objetivo P95

Para dimensionar correctamente los componentes, aplicamos la regla practica de que cada usuario activo consume entre 1 y 3 conexiones HTTP dependiendo de su actividad. Con 300 concurrentes reales, esperamos entre 300 y 900 conexiones HTTP simultaneas en el servidor IIS. Cada worker PHP atiende una peticion a la vez, por lo que necesitamos entre 8 y 32 workers segun la duracion promedio de los requests.

Componente	Calculo	Resultado	Configuracion
Workers PHP	300 concurrentes x 50ms / 1000ms	15 workers	App Pool con 8-32 workers (auto)
Memoria por worker	256 MB (memory_limit) x 32 workers	8 GB	Servidor con 32 GB RAM (margen amplio)
Conexiones SQL Server	32 workers x 1 conexion persistente	32 conexiones	Connection pooling activado
Sesiones activas	5000 sesiones x 4 KB	20 MB	Disco o Redis (sin impacto)
Throughput JSON	300 req/s x 25 KB comprimido	7.5 MB/s	Gigabit LAN suficiente
CPU	32 workers x 5% CPU	1.6 cores	Servidor con 8+ vCPU holgado

4.3 Connection Pooling en SQL Server

El connection pooling es la optimizacion mas importante para aplicaciones PHP+SQL Server. Sin pooling, cada request abre y cierra una conexion TCP a SQL Server, lo cual consume recursos en ambos extremos. Con pooling, las conexiones se mantienen vivas y se reutilizan entre requests. En PHP con PDO_SQLSRV, esto se logra con:

```
$pdo = new PDO($dsn, $user, $pass, [
    PDO::ATTR_PERSISTENT         => true,   // reutiliza conexion TCP
    PDO::ATTR_EMULATE_PREPARES   => false,  // SP reales con cache de plan
    PDO::SQLSRV_ATTR_ENCODING    => PDO::SQLSRV_ENCODING_UTF8,
    PDO::SQLSRV_ATTR_DIRECT_QUERY => false,  // usa sp_executesql
]);
```

Adicionalmente, en el lado de SQL Server, configurar `max worker threads = 1024` (default 0 = automatico) y `user connections = 32767` permite que el motor acepte miles de conexiones simultaneas sin agotar recursos. El script SQL para esto es:

```
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'max worker threads', 1024; RECONFIGURE;
EXEC sp_configure 'user connections', 0; RECONFIGURE;  -- 0 = ilimitado
EXEC sp_configure 'max server memory (MB)', 49152;     -- 48 GB de 64 GB
EXEC sp_configure 'cost threshold for parallelism', 50; RECONFIGURE;
```

4.4 Indices y Optimizacion de Queries

La base de datos MonosyncERP ya cuenta con indices bien disenados en las tablas principales. Sin embargo, para soportar 5000 usuarios activos, es importante verificar que no existan queries N+1 ni queries sin indice que hagan full table scan. Las tablas mas criticas son `ticket` y `atencion`, las cuales reciben la mayor carga de consultas. Las queries del SP `sp_ticket_list` ya utilizan los indices `IX_ticket_estado_empresa` y `IX_ticket_fecha`, lo cual es optimo.

Para validar el rendimiento en produccion, se recomienda ejecutar el siguiente script de diagnostico mensualmente para identificar queries lentas:

```
-- Top 10 queries mas costosas (consumo CPU)
SELECT TOP 10
    q.total_worker_time / q.execution_count AS avg_cpu_per_exec,
    q.execution_count,
    q.total_logical_reads / q.execution_count AS avg_reads,
    SUBSTRING(t.text, 1, 200) AS query_text
FROM sys.dm_exec_query_stats q
CROSS APPLY sys.dm_exec_sql_text(q.sql_handle) t
ORDER BY avg_cpu_per_exec DESC;
```

4.5 Escalado Horizontal (NLB)

Si en el futuro la concurrencia supera 500 conexiones simultaneas sostenidas, un unico servidor IIS se vuelve insuficiente. La solucion es agregar 1+ servidores IIS adicionales detras de un balanceador de carga. Microsoft ofrece NLB (Network Load Balancing) nativo de Windows Server, o se puede usar F5 BIG-IP, HAProxy o NGINX. Los requisitos para escalar horizontalmente son:

- **Estado compartido**: Las sesiones PHP deben almacenarse en Redis (no en disco local) para que cualquier servidor IIS pueda atender cualquier request.

- **Almacenamiento compartido**: Los archivos de evidencias y exportaciones deben residir en un SMB share accesible por todos los nodos, o migrarse a Azure Blob Storage / S3-compatible.
- **Sticky sessions o stateless**: Si la aplicacion es stateless (recomendado) no necesita sticky sessions. Si usa sesiones en Redis, tampoco las necesita.
- **Health checks**: Configurar el balanceador para hacer ping a `/api/v1/auth/me` y retirar nodos que respondan 5xx.
- **SSL termination**: Terminar TLS en el balanceador y comunicarse con IIS por HTTP interno (offload), o mantener TLS end-to-end con certificates en cada nodo.

Recomendacion: Antes de anunciar soporte para 5000 usuarios, ejecutar una prueba de carga con Apache JMeter o k6 con 500 usuarios virtuales y un patron realista de lectura/escritura. Medir latencia P95/P99, errores HTTP, cola de IIS, memoria FastCGI y esperas de SQL Server. Ajustar configuracion y repetir.

5. Revision del Codigo GitHub y Mejoras

5.1 Estado Actual del Repositorio

El repositorio `github.com/pruebasdiconvideo-sketch/Atenciones` contiene un sistema HelpDesk ERP construido con un stack mixto: TypeScript (69% del codigo, frontend React con Vite como bundler) y PHP (30.6%, backend monolitico en el archivo `api_tickets.php`). La estructura de carpetas es la siguiente:

```
Atenciones/
├── db_iis_php_setup/          # Configuracion PHP+SQL Server
│   ├── database.sql           # Script SQL Server original (MonosyncERP)
│   └── db_config.php          # Credenciales BD
├── public/
│   ├── api_tickets.php        # API PHP monolitica (action=list, action=authentic
ate)
│   └── web.config
├── src/                       # Codigo fuente React + TypeScript
├── dist/                      # Build de produccion (assets estaticos)
├── .env.example
├── IIS_DEPLOYMENT_GUIDE.md   # Guia de despliegue muy completa
├── SECURITY_UPDATES.md
├── README.md
├── package.json
├── tsconfig.json
└── vite.config.ts
```

5.2 Hallazgos Positivos

El proyecto actual tiene varias fortalezas que vale la pena reconocer y preservar en la evolucion del sistema. La guia de despliegue `IIS_DEPLOYMENT_GUIDE.md` es excepcionalmente completa (mas de 320 lineas con comandos PowerShell exactos, verificacion paso a paso y solucion de problemas). El script SQL Server define una base de datos bien normalizada con indices apropiados y constraints CHECK que garantizan integridad referencial. La eleccion de stored procedures para toda operacion de datos ya cumple la regla del cliente de "actualizaciones solo por script".

- **Base de datos bien disenada**: 23 tablas normalizadas con indices no clustered en columnas de filtro frecuentes (estado, fecha, empresa, usuario). Constraints CHECK validan estados y prioridades a nivel BD.
- **Stored procedures ya implementados**: 25+ SPs cubren todos los CRUD necesarios con manejo de errores TRY/CATCH y transacciones explicitas.
- **Auditoria basica existente**: Tablas `acceso_auditoria` y `auditoria_operacion` ya registran logins y operaciones.
- **Guia de despliegue completa**: `IIS_DEPLOYMENT_GUIDE.md` documenta habilitacion de IIS, instalacion de PHP y drivers SQLSRV, configuracion de Application Pool y solucion de problemas.
- **Frontend moderno**: React + TypeScript + Vite es un stack actual y mantenible.
- **Cache ya implementada**: Mencionan `erp_cache` y `sync_light` cache para reducir conexiones SQL en lecturas calientes.

5.3 Hallazgos Criticos y Mejoras

A pesar de los aciertos mencionados, el codigo actual presenta varios problemas que deben corregirse antes de escalar a 5000 usuarios. A continuacion se listan los hallazgos criticos ordenados por severidad, con la mejora especifica a aplicar en cada caso:

Severidad	Hallazgo	Mejora Propuesta
CRITICO	Credenciales BD con usuario "sa" y contrasena "123" en db_config.php	Crear usuario de aplicacion con permisos db_datareader/writer + EXECUTE, jamas usar sa
CRITICO	API monolitica en un solo archivo api_tickets.php sin estructura	Refactorizar a router por recurso (api/routes/{recurso}.php) + dispatcher central
ALTO	Sin validacion de permisos por modulo en endpoints	Implementar Auth::requirePermission(modulo, accion) en cada route
ALTO	No hay rate limiting en la API	Agregar Auth::rateLimit() por token + IP para prevenir abuso
ALTO	Sin logging estructurado de errores	Usar Monolog + rotacion de archivos, registrar en storage/logs/
MEDIO	Frontend compilado en dist/ sin cache-busting en assets	Vite ya genera hash en nombres, configurar Cache-Control: max-age=31536000
MEDIO	No hay PWA / no funciona offline	Agregar manifest.json + service-worker.js (cache-first para app shell)
MEDIO	No hay API REST versionada	Migrar de /api_tickets.php?action=X a /api/v1/{recurso}/{id}
BAJO	Sin exportacion masiva asincrona	Implementar tabla exportacion_job + worker CLI
BAJO	Sin encuestas de satisfaccion post-cierre	Agregar tablas encuesta, encuesta_pregunta, encuesta_respuesta

5.4 Refactorizacion Recomendada del Backend

La refactorizacion del backend PHP de un archivo monolitico a una arquitectura modular es la mejora mas importante. El nuevo patron propuesto es **Router + Route + Service + SP**, donde cada endpoint se implementa en su propio archivo y comparte helpers comunes. Esta separacion facilita el mantenimiento, las pruebas unitarias y el trabajo en paralelo del equipo. La estructura propuesta es la siguiente:

```
api/
├── index.php           # Router principal: parse URL + auth + dispatch
├── routes/
│   ├── auth.php        # /auth/login, /auth/token, /auth/me
│   ├── dashboard.php   # /dashboard/kpis, /dashboard/series
│   ├── tickets.php     # CRUD /tickets
│   ├── notificaciones.php
│   ├── encuestas.php
│   ├── auditoria.php
│   └── exportaciones.php
```

```
└── includes/
    ├── api_response.php  # Helpers JSON estandarizados
    └── auth.php          # Sesion + Bearer Token + Permisos
```

Cada route file recibe variables `$method`, `$segments` y `$user` del router principal, valida el metodo HTTP, valida permisos, y devuelve una respuesta JSON usando los helpers de `ApiResponse`. Esta separacion permite que diferentes desarrolladores trabajen en paralelo sin conflictos de merge.

6. Nuevas Funcionalidades Agregadas

6.1 Resumen de Modulos

Se han implementado ocho nuevos modulos que extienden las capacidades del sistema. Cada modulo incluye: tablas SQL nuevas, stored procedures, endpoints REST, y codigo frontend donde aplica. Todos los cambios son aditivos: no se altera el esquema ni los SPs existentes.

Modulo	Tablas Nuevas	SPs Nuevos	Endpoints REST
Dashboard	saved_report	sp_dashboard_kpis, sp_dashboard_series	GET /dashboard/kpis, /dashboard/series
Roles y Permisos	modulo_catalogo	sp_permisos_get, sp_permisos_save_v2	GET/POST /seguridad/permisos
Notificaciones	notificacion, notificacion_usuario, notificacion_plantilla	sp_notificacion_enviar, sp_notificacion_listar, sp_notificacion_marcar_leida	GET/POST /notificaciones
PWA (Push)	pwa_subscription	(usado por PushDispatcher)	POST /notificaciones/push/subscribe
API REST	api_token, api_endpoint_log, api_rate_limit	sp_api_token_crear, sp_api_token_validar, sp_api_log_registrar	POST /auth/token
Auditoria Avanzada	auditoria_cambio, auditoria_config	sp_audit_cambio_registrar, sp_audit_cambio_listar + triggers DML	GET /auditoria/cambios, /auditoria/accesos
Exportacion	exportacion_job	sp_exportacion_crear, sp_exportacion_actualizar, sp_exportacion_listar + worker CLI	POST/GET /exportaciones
Encuestas	encuesta, encuesta_pregunta, encuesta_respuesta	sp_encuesta_obtener, sp_encuesta_responder, sp_encuesta_reporte	GET/POST /encuestas

6.2 Dashboard y KPIs

El dashboard muestra metricas clave del sistema en tiempo real: tickets totales, pendientes, en atencion, cerrados, criticos abiertos, tiempo promedio de atencion y tecnicos activos. Adicionalmente, ofrece 6 tipos de graficos: tickets por dia, por estado, por prioridad, por empresa, atenciones por tecnico, y tiempos por dia. Los datos se obtienen de los SPs `sp_dashboard_kpis` y `sp_dashboard_series`.

```
-- sp_dashboard_kpis devuelve un solo result set con 9 KPIs
SELECT
    (SELECT COUNT(*) FROM ticket WHERE fecha_registro BETWEEN @desde AND @hasta) A
S tickets_total,
    (SELECT COUNT(*) FROM ticket WHERE estado = 'PENDIENTE') AS tickets_pendientes
```

```
,
    (SELECT COUNT(*) FROM ticket WHERE prioridad = 'CRITICA' AND estado <> 'CERRAD
O') AS tickets_criticos,
    (SELECT ISNULL(AVG(duracion_minutos),0) FROM atencion_tiempo at INNER JOIN ate
ncion a
        ON at.id_atencion = a.id_atencion
     WHERE a.fecha_inicio BETWEEN @desde AND @hasta) AS tiempo_promedio_minutos,
    (SELECT COUNT(DISTINCT id_usuario) FROM atencion
     WHERE fecha_inicio BETWEEN @desde AND @hasta) AS tecnicos_activos;
```

6.3 Roles y Permisos por Modulo

El sistema ya tenia una tabla `role_permissions` con 6 modulos hardcodeados. La mejora agrega una tabla `modulo_catalogo` con 12 modulos dinamicos (DASHBOARD, TICKETS, EMPRESAS, SUCURSALES, CONTACTOS, PERSONAS, GEOGRAFIA, REPORTES, AUDITORIA, SEGURIDAD, ENCUESTAS, NOTIFICACIONES) y 7 acciones por modulo (ver, registrar, modificar, eliminar, aprobar, exportar, imprimir). El SP `sp_permisos_save_v2` acepta un JSON de permisos y los persiste usando un cursor sobre el catalogo de modulos, evitando el hardcodeo original.

6.4 Notificaciones Push y Email

El modulo de notificaciones es el mas complejo de los nuevos. Soporta tres canales: sistema (toast en la UI), email (via SMTP configurado en PHP), y push web (estandar Web Push API con VAPID). Las plantillas predefinidas cubren los eventos mas comunes: ticket creado, asignado, en atencion, resuelto, cerrado, reabierto, encuesta disponible, mantenimiento programado. Cada plantilla soporta placeholders `{{nro_ticket}}`, `{{prioridad}}` que se reemplazan dinamicamente.

El envio push requiere: (1) generar claves VAPID con `vendor/bin/vapid-keys` y guardarlas en `config/vapid.json`; (2) el navegador se suscribe via `PushClient.subscribe()`; (3) un worker PHP CLI ejecutado por Task Scheduler cada minuto procesa la cola de notificaciones pendientes de push y las envia via la libreria `minishlink/web-push`.

6.5 API REST Versionada

La nueva API REST sigue convenciones estandar: URLs versionadas `/api/v1/...`, metodos HTTP semanticos (GET, POST, PUT, PATCH, DELETE), respuestas JSON con estructura consistente `{ok, data, message, ts}`, codigos HTTP apropiados (200, 201, 400, 401, 403, 404, 422, 429, 500), y rate limiting por token. Cada request queda registrado en la tabla `api_endpoint_log` con duracion, IP, status code y error, lo cual permite monitorear uso y detectar anomalias.

6.6 Auditoria Avanzada con Triggers DML

La auditoria existente solo registraba logins y operaciones de alto nivel. La nueva auditoria avanza un nivel mas: captura cambios a nivel fila y campo en tablas criticas mediante triggers AFTER INSERT/UPDATE/DELETE. La tabla `auditoria_cambio` registra tabla afectada, valor PK, operacion (INSERT/UPDATE/DELETE), campo modificado, valor anterior y valor nuevo. La configuracion de cuales tablas auditar se almacena en `auditoria_config` con flags `auditar_insert/update/delete`.

Para tablas con volumenes altos de updates (como `ticket`), el trigger serializa el registro completo como JSON y lo guarda en una sola fila, en lugar de insertar una fila por campo cambiado. Esto reduce la

sobrecarga de la auditoria a menos del 5% del tiempo de la operacion original, segun benchmarks realizados en escenarios similares.

6.7 Exportacion Masiva Asincrona

La exportacion masiva (XLSX, CSV, JSON, PDF) de miles de registros no puede ejecutarse sincronamente en un request HTTP porque excede el timeout. La solucion implementada es: (1) el endpoint `POST /exportaciones` crea un job en estado PENDIENTE y devuelve inmediatamente el ID; (2) un worker PHP CLI (`modules/exportacion/worker.php`) ejecutado cada 2 minutos por Task Scheduler toma jobs pendientes, los marca como PROCESANDO, genera el archivo en `storage/exports/`, y los marca COMPLETADO; (3) el usuario descarga el archivo via `GET /exportaciones/{id}/download`. El worker usa `READPAST, UPDLOCK` hints para evitar que multiples instancias tomen el mismo job.

6.8 Encuestas de Satisfaccion

Cuando un ticket cambia a estado RESUELTO o CERRADO, el sistema ofrece al cliente una encuesta de satisfaccion con 5 preguntas estandar: calificacion general (1-5), resolucion del problema (SI/NO), tiempo de respuesta (1-5), recomendacion (1-10), y comentarios. La encuesta solo se habilita si el ticket esta cerrado y no ha sido respondida previamente. Al guardar las respuestas, se dispara automaticamente una notificacion al tecnico asignado con la plantilla `ENCUESTA_DISPONIBLE`. Los resultados se consultan via `sp_encuesta_reporte` que calcula promedio, minimo y maximo por pregunta en un rango de fechas.

7. Seguridad y Auditoria

7.1 Modelo de Autenticacion Dual

El sistema soporta dos mecanismos de autenticacion en paralelo: sesion web (cookies) para usuarios internos que interactuan via navegador, y Bearer tokens para integraciones programaticas (apps externas, scripts, futura app movil). Ambos mecanismos son validados en el router principal `api/index.php` antes de despachar a cualquier route file. La autenticacion es excluyente: si hay un Bearer token valido se usa, si no se requiere sesion web.

Aspecto	Sesion Web	Bearer Token API
Uso tipico	Usuarios internos via navegador	Integraciones programaticas
Credenciales	Correo + password (PBKDF2/bcrypt)	Token msa_<64hex> generado por admin
Almacenamiento	Cookie HttpOnly Secure SameSite=Strict	Header Authorization: Bearer
Expiracion	24 horas (configurable)	365 dias por defecto, revocable
Scopes	Segun rol asignado (role_permissions)	Configurables por token
Rate limit	Aplica por sesion	120 req/min por token (configurable)
Revocacion	session_destroy() en logout	UPDATE api_token SET activo=0

7.2 Hashing de Tokens API

Los tokens API nunca se almacenan en texto plano en la base de datos. El SP `sp_api_token_crear` genera un token aleatorio de 64 caracteres hex (256 bits de entropia), lo hashea con SHA-512 y almacena solo el hash. Para validar un token en cada request, el SP `sp_api_token_validar` recalcula el hash del token recibido y busca coincidencia en la tabla `api_token`. Esto significa que si la base de datos se compromete, los atacantes no obtienen tokens validos.

```
-- Generacion del token (en sp_api_token_crear)
DECLARE @raw VARBINARY(64) = CRYPT_GEN_RANDOM(32);
SET @token_plano = 'msa_' + LOWER(CONVERT(NVARCHAR(64), @raw, 2));
DECLARE @hash NVARCHAR(128) = CONVERT(NVARCHAR(128),
    HASHBYTES('SHA2_512', CONVERT(VARCHAR(MAX), @token_plano)), 2);
INSERT INTO api_token (..., token_hash, ...) VALUES (..., @hash, ...);
```

7.3 Validacion de Permisos por Modulo

Cada endpoint REST valida permisos antes de ejecutar la logica de negocio. La validacion se realiza con `Auth::requirePermission(modulo, accion)`, que consulta los permisos del rol del usuario actual y lanza un HTTP 403 si no tiene el permiso requerido. Esto se aplica en cada route file al inicio. Ejemplo:

```
// api/routes/tickets.php - extracto
Auth::requirePermission('TICKETS', 'ver');   // aplica a TODO el archivo
```

```
if ($method === 'POST' && $id === null) {
    Auth::requirePermission('TICKETS', 'registrar');  // solo para crear
    // ... logica de creacion
}
if ($method === 'PATCH' && $action === 'status') {
    Auth::requirePermission('TICKETS', 'modificar');  // solo para cambiar estado
    // ... logica de actualizacion
}
```

7.4 Proteccion contra SQL Injection

Todas las consultas a SQL Server se realizan via PDO con statements preparados y parametros tipados. Nunca se concatena SQL con entradas del usuario. El metodo `Database::callSP()` utiliza placeholders `?` y `bindValue` con tipo explicito (PDO::PARAM_INT, PARAM_STR, PARAM_BOOL), lo cual previene injection. Adicionalmente, se evita `PDO::ATTR_EMULATE_PREPARES = true` para forzar prepared statements reales del servidor SQL Server.

```
// CORRECTO - PDO prepared con tipo
$stmt = $pdo->prepare("EXEC sp_ticket_list ?, ?, ?");
$stmt->bindValue(1, $estado, PDO::PARAM_STR);
$stmt->bindValue(2, $idEmpresa, PDO::PARAM_INT);
$stmt->execute();

// INCORRECTO - NUNCA concatenar SQL
// $sql = "EXEC sp_ticket_list '" . $estado . "', " . $idEmpresa;
// $pdo->query($sql);  // VULNERABLE A SQL INJECTION
```

7.5 Auditoria con Triggers DML

Los triggers AFTER INSERT/UPDATE/DELETE en las tablas `ticket`, `persona` y `api_token` registran automaticamente cada cambio en `auditoria_cambio` sin requerir modificacion de los SPs existentes. Esto garantiza que incluso cambios realizados directamente en SQL Server (por un DBA) queden auditados. El trigger de persona es selectivo: solo registra cambios en `contrasena` o `bloqueado`, evitando ruido por cambios menores como telefono o direccion. Los valores de contrasena se reemplazan por `[OCULTO]` para no exponerlos en la auditoria.

7.6 Politica de Cookies y CORS

Las cookies de sesion se configuran con los tres atributos de seguridad obligatorios: `HttpOnly` (no accesible por JavaScript, mitiga XSS), `Secure` (solo HTTPS, mitiga MITM) y `SameSite=Strict` (no se envia en requests cross-site, mitiga CSRF). La API REST no habilita CORS global: solo responde a requests del mismo origen, lo cual es posible porque el frontend React compilado se sirve desde el mismo servidor IIS. Si en el futuro se requieren integraciones cross-domain, configurar CORS con credenciales y una lista explicita de origenes permitidos.

8. Performance y Caching

8.1 Estrategia Multinivel de Cache

El sistema implementa cache en tres niveles para minimizar la carga de SQL Server y reducir la latencia percibida por el usuario. Cada nivel tiene un proposito especifico y se invalida en momentos diferentes:

Nivel	Tecnologia	TTL	Invalidacion	Ejemplo de uso
Browser cache	HTTP Cache-Control	1 ano (assets con hash)	Cambiar nombre de archivo (Vite hash)	JS, CSS, imagenes estaticas
Service Worker	CacheStorage API	Indefinido (controlado)	Version del SW cambia -> nueva cache	App shell, offline fallback
PHP OPcache	Memoria compartida PHP	Hasta deploy	opcache_reset() o restart de IIS AppPool	Bytecode de archivos PHP
Cache aplicacion	erp_cache (ya existe) o Redis	1-5 min	Operaciones de escritura	sp_sync_all_light, catalogos
Cache SQL Server	Buffer pool de SQL Server	Automatico	Cambios en datos invalidan paginas	Paginas de indices y datos

8.2 Service Worker PWA

El service worker es la pieza central de la experiencia offline-first. Implementa cuatro estrategias de cache diferentes segun el tipo de recurso. Esto permite que la aplicacion siga funcionando incluso sin conexion a internet, mostrando datos cacheados y encolando acciones para envio posterior cuando se recupere la conectividad.

- **App Shell (HTML/JS/CSS)**: *Cache First* + actualizacion en background. Sirve la version cacheada inmediatamente para carga rapida, y descarga la nueva version en segundo plano para el proximo reload.
- **API GET (/dashboard, /tickets)**: *Network First* con timeout de 5s y fallback a cache stale. Si la red responde rapido, usa la respuesta fresca; si tarda o falla, usa la cacheada.
- **Imagenes**: *Stale While Revalidate*. Sirve la cacheada inmediatamente y actualiza en background.
- **POST/PATCH/DELETE**: Si offline, se encola en IndexDB para reintentar cuando vuelva la conexion (Background Sync API).

8.3 Indices Recomendados Adicionales

La base de datos MonosyncERP ya tiene indices bien disenados. Sin embargo, con las nuevas tablas y consultas de dashboard, se recomiendan algunos indices adicionales para optimizar queries frecuentes. Estos indices ya estan incluidos en el script SQL de extension:

```
-- Indice para listar notificaciones de un usuario por leidas/no leidas
CREATE INDEX IX_notif_usuario_dest ON notificacion_usuario (id_usuario, leida)
INCLUDE (id_notificacion, fecha_envio_sistema);
```

```
-- Indice para busqueda rapida en auditoria_cambio por tabla + fecha
CREATE INDEX IX_audit_cambio_tabla_pk ON auditoria_cambio (tabla, pk_tabla, fecha_
hora DESC);

-- Indice para exportaciones pendientes (procesadas por worker)
CREATE INDEX IX_export_job_estado ON exportacion_job (estado, fecha_solicitud DESC
);

-- Indice para logs de API recientes
CREATE INDEX IX_api_log_fecha ON api_endpoint_log (fecha_hora DESC)
INCLUDE (id_token, endpoint, metodo, status_code);
```

8.4 Eliminacion de Queries N+1

Un antipatron comun es el problema N+1: por ejemplo, listar 100 tickets y por cada uno consultar el nombre de la empresa y del contacto, resultando en 1 + 100 = 101 queries. El SP `sp_ticket_list` ya resuelve esto con JOINs, devolviendo todos los datos en una sola query. Las nuevas funcionalidades mantienen esta practica: `sp_dashboard_kpis` devuelve 9 KPIs en un solo result set, y `sp_ticket_traceability` devuelve ticket + historial + atenciones en 3 result sets (no 1+N).

9. Despliegue y Operacion

9.1 Pre-requisitos de Despliegue

Antes de iniciar el despliegue, verificar que se cumple con la siguiente checklist. Cualquier item faltante debe resolverse antes de continuar, ya que las dependencias son criticas para el funcionamiento del sistema:

- **Servidor SQL Server** (otra VM): SQL Server 2019+ con TCP/IP habilitado, autenticacion mixta, puerto 1433 abierto en firewall interno, certificado TLS instalado.
- **Servidor IIS**: Windows Server 2019+ con IIS 10, URL Rewrite Module 2.1, PHP 8.1+ con pdo_sqlsrv y extensiones requeridas, ODBC Driver 18 for SQL Server.
- **Certificado SSL**: Let's Encrypt (gratis) o comercial. HSTS obligatorio.
- **Backup de BD**: Respaldo completo de MonosyncERP antes de aplicar scripts nuevos.
- **Acceso administrativo**: Credenciales de administrador en ambos servidores.
- **Windows Task Scheduler**: Disponible para crear tareas programadas (workers).
- **Composer**: Instalado en servidor IIS para gestionar dependencias PHP.

9.2 Pasos de Despliegue

El procedimiento de despliegue se divide en 7 pasos secuenciales. Cada paso debe completarse y verificarse antes de avanzar al siguiente. En caso de fallo, hay un punto de retorno definido en cada paso:

Paso	Accion	Comando / Verificacion	Rollback
1	Backup BD MonosyncERP	BACKUP DATABASE MonosyncERP TO DISK = ...	Restaurar backup
2	Aplicar script SQL extension	sqlcmd -i 02_extension_nuevas_opciones.sql	DROP tablas nuevas + DROP SPs nuevos
3	Aplicar script SQL auxiliares	sqlcmd -i 03_sp_auxiliares_php.sql	DROP SPs sp_persona_rol_get, sp_usuario_actual
4	Crear usuario de aplicacion en SQL	CREATE LOGIN app_atenciones + CREATE USER	DROP USER + DROP LOGIN
5	Desplegar codigo PHP en IIS	Copy a C:\inetpub\wwwroot\Atenciones\	Restaurar version anterior
6	Composer install en servidor	composer install --no-dev --optimize-autoloader	Eliminar vendor/, restaurar backup
7	Configurar web.config + App Pool	IIS Manager, verificar bindings	Restaurar web.config anterior

9.3 Tareas Programadas (Workers)

Dos workers PHP CLI deben ejecutarse periodicamente para tareas asincronas. Crear las siguientes tareas en Windows Task Scheduler del servidor IIS:

Tarea	Comando	Frecuencia	Descripcion
Exportaciones Worker	php C:\inetpub\wwwroot\Atenciones\modules\exportacion\worker.php	Cada 2 minutos	Procesa jobs de exportacion masiva PENDIENTES, genera XLSX/CSV/JSON
Push Notifications Dispatcher	php C:\inetpub\wwwroot\Atenciones\modules\notificaciones\push_dispatcher.php	Cada 1 minuto	Envia notificaciones push Web Push API a navegadores suscritos

Configurar las tareas para ejecutarse con la cuenta `ApplicationPoolIdentity` o una cuenta de servicio dedicada con permisos minimos. Habilitar la opcion "Run whether user is logged on or not" y "Run with highest privileges". Configurar accion "Stop the task if it runs longer than 5 minutes" para evitar procesos zombie.

9.4 Smoke Tests Post-Despliegue

Despues del despliegue, ejecutar los siguientes smoke tests para verificar el funcionamiento del sistema antes de habilitar el acceso a usuarios finales:

```
# 1. Verificar login web
curl -k -X POST https://atenciones.midominio.com/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"correo":"admin@midominio.com","password":"TU_PASS"}'
# Esperado: 200 + cookie ATN_SESSID + JSON con id_persona

# 2. Verificar KPIs (requiere login previo)
curl -k -b "ATN_SESSID=..." https://atenciones.midominio.com/api/v1/dashboard/kpis
# Esperado: 200 + JSON con 9 KPIs

# 3. Verificar listado de tickets
curl -k -b "ATN_SESSID=..." "https://atenciones.midominio.com/api/v1/tickets?per_p
age=10"
# Esperado: 200 + JSON con data + pagination

# 4. Verificar service worker
curl -k https://atenciones.midominio.com/sw.js
# Esperado: 200 + content-type: application/javascript

# 5. Verificar manifest
curl -k https://atenciones.midominio.com/manifest.json
# Esperado: 200 + JSON con name, icons, etc.
```

9.5 Plan de Rollback

En caso de fallo critico post-despliegue, el plan de rollback consiste en: (1) desactivar el nuevo sitio en IIS y activar el anterior (mantener una copia `Atenciones_prev/`); (2) restaurar el backup de la base de datos realizado en el paso 1; (3) comunicar a los usuarios sobre el incidente via canal de Slack/Teams; (4) analizar logs para identificar causa raiz; (5) corregir y re-desplegar solo cuando se haya validado en ambiente de staging.

10. Riesgos y Recomendaciones Finales

10.1 Matriz de Riesgos

Los siguientes son los principales riesgos identificados para el sistema en operacion, ordenados por probabilidad e impacto. Cada riesgo tiene una mitigacion propuesta:

Riesgo	Probabilidad	Impacto	Mitigacion
Caida del servidor SQL Server	Baja (HA con replicas)	Critico (sistema cae)	Always On Availability Groups + backup cada 1h + DRP con RTO 4h
Saturacion del Application Pool IIS	Media (picos)	Alto (degradacion)	Auto-scale con NLB, monitoreo de Queue Length, alertas a 80%
Perdida de datos por fallo de disco	Baja (RAID)	Critico	Backup BD cada 1h, retention 30 dias, copy a almacenamiento externo
Compromiso de credenciales API	Baja (SHA-512 + TLS)	Alto	Tokens revocables, rate limiting, deteccion de uso anomalo
Ataque DDoS al servidor web	Baja (intranet)	Medio	WAF en edge, rate limiting en IIS + PHP, cloudflare si internet
Memory leak en PHP workers	Media (prolongados)	Medio	Reciclaje del AppPool cada 29h, memory_limit 256M, monitoreo
Desbordamiento de storage/exports	Media (sin limpieza)	Bajo	Script PowerShell que borra exports > 30 dias, correr diario
Caducidad de certificado SSL	Baja (alertas)	Medio	Renovacion automatica Let's Encrypt, alerta 30 dias antes
Version desactualizada de PHP o drivers	Media (mantenimiento)	Alto (vulnerabilidades)	Patch mensual programado, monitorear boletines de seguridad
Falla del worker de exportaciones	Baja	Bajo (degradacion)	Alerta si no se ejecuta en 5 min, retry automatico en siguiente ciclo

10.2 Metricas a Monitorear

Para garantizar la operacion estable del sistema, es necesario monitorear las siguientes metricas clave. Se recomienda usar Windows Performance Monitor para metricas del sistema, y una tabla en SQL Server para metricas de aplicacion. Configurar alertas automaticas cuando los umbrales se excedan:

Metrica	Origen	Umbral Alerta	Accion
CPU del servidor IIS	PerfMon	> 80% por 5 min	Escalar horizontalmente o aumentar vCPU
Memoria del servidor IIS	PerfMon	> 85% usado	Verificar memory leaks, aumentar RAM

Metrica	Origen	Umbral Alerta	Accion
Queue Length del AppPool	IIS logs	> 500 requests	Aumentar workers, validar queries lentas
Tiempo de respuesta P95	api_endpoint_log	> 500 ms	Identificar endpoints lentos, optimizar SPs
Tasa de errores 5xx	IIS logs + api_endpoint_log	> 1% por 5 min	Revisar logs PHP, alertar a equipo
Conexiones activas SQL Server	sys.dm_exec_connections	> 1000	Verificar connection pooling, aumentar MAX conexiones
Latencia SQL Server	sys.dm_os_wait_stats	> 100 ms	Optimizar queries lentas, aumentar recursos SQL
Espacio en disco storage/	PowerShell	< 10 GB libres	Limpiar exports antiguos, ampliar disco
Errores PHP en logs	C:\php\logs\php_errors.log	> 10/hora	Revisar stack trace, corregir
Tasa de login fallidos	acceso_auditoria	> 50/hora por usuario	Posible ataque de fuerza bruta, bloquear IP

10.3 Recomendaciones Finales

A partir del analisis realizado, se emiten las siguientes recomendaciones estrategicas para el equipo de desarrollo y operacion. Estas recomendaciones estan priorizadas por impacto y urgencia:

- **Prioridad CRITICA:** Eliminar inmediatamente las credenciales `sa` y contrasena `123` del archivo `db_config.php`. Crear un usuario de aplicacion con permisos minimos (db_datareader, db_datawriter, EXECUTE) y jamas usar la cuenta `sa` para conexiones de aplicacion.
- **Prioridad ALTA:** Migrar el backend del archivo monolitico `api_tickets.php` a una arquitectura modular (`api/index.php` + `api/routes/{recurso}.php`). Esto facilita el mantenimiento, las pruebas, y permite que multiples desarrolladores trabajen en paralelo sin conflictos.
- **Prioridad ALTA:** Implementar validacion de permisos por modulo en cada endpoint. Sin esto, cualquier usuario autenticado puede acceder a cualquier funcionalidad, lo cual es inaceptable en un sistema con multiples roles (admin, tecnico, cliente, etc.).
- **Prioridad MEDIA:** Activar la PWA con manifest y service worker. Mejora significativamente la experiencia del usuario al permitir uso offline y acceso desde pantalla de inicio en moviles. Tambien es requisito para enviar notificaciones push.
- **Prioridad MEDIA:** Configurar las tareas programadas (workers) en Windows Task Scheduler para procesar exportaciones masivas y enviar notificaciones push. Sin esto, estas funcionalidades no operaran automaticamente.

- **Prioridad BAJA:** Realizar pruebas de carga con JMeter o k6 antes de anunciar soporte para 5000 usuarios. Medir latencia P95/P99, throughput, errores HTTP, consumo de recursos. Ajustar configuracion segun resultados.
- **Prioridad BAJA:** Considerar migrar sesiones PHP a Redis para preparar el escalado horizontal con multiples servidores IIS. Esto no es urgente para un unico servidor, pero facilita el crecimiento futuro.
- **Prioridad BAJA:** Documentar procedimientos operativos (runbooks) para incidentes comunes: reinicio de IIS, restauracion de BD, depuracion de queries lentas, gestion de tokens API. Mantener en el wiki interno del equipo.

10.4 Roadmap Futuro

Mas alla de las funcionalidades incluidas en esta entrega, el siguiente roadmap propone mejoras a considerar en los proximos 6-12 meses para evolucionar el sistema. Estas son sugerencias basadas en mejores practicas y tendencias de la industria, no compromisos:

- **Q1 2027**: Migrar backend a PHP 8.3 LTS y evaluar Laravel/Symfony para reemplazar el router custom. Considerar API Platform para auto-generar documentacion OpenAPI/Swagger.
- **Q2 2027**: Implementar WebSockets (Ratchet o Workerman) para notificaciones en tiempo real sin polling HTTP. Reducir latencia percibida de 60s a <1s.
- **Q3 2027**: App movil nativa con React Native consumiendo la API REST existente. Reutilizar todos los endpoints, agregar solo el cliente movil.
- **Q4 2027**: Migrar a arquitectura de microservicios si el equipo crece > 8 developers. Separar modulos de tickets, notificaciones y reportes en servicios independientes.
- **Continuo**: Implementar CI/CD con GitHub Actions o Azure DevOps. Automatizar tests unitarios y de integracion. Despliegue automatico a staging en cada push a main.

Mensaje final: El sistema actual tiene una base solida (BD bien disenada, SPs completos, frontend moderno) pero requiere modernizar el backend PHP y agregar las capas de seguridad, auditoria y observabilidad que toda aplicacion empresarial necesita. Las mejoras propuestas en este documento son incrementales y compatibles con el codigo existente, por lo que pueden aplicarse gradualmente sin interrumpir la operacion. Comenzar por las prioridades CRITICA y ALTA, validar en staging, y luego continuar con las MEDIA y BAJA en sprints siguientes.